

Roteiro da Oficina 4: Algoritmos para Resolução do Cubo Mágico

Etapa 1 - Leitura do cubo mágico

1. Estrutura do Cubo

O cubo mágico é um quebra-cabeça composto por 27 peças e 54 adesivos. Podemos classificar essas peças em três tipos, são elas:

- centros (*centers*)
- meios (*edges*)
- quinas (*corners*)

Na Figura 1 podemos observar melhor a localização de cada uma delas.

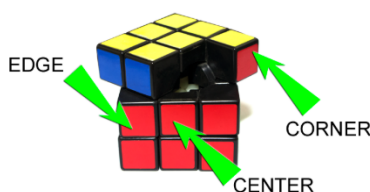


Figura 1: Tipos de Peças do Cubo

Observando o cubo, podemos ver que ele tem 6 centros, 12 meios e 8 quinas, além de uma peça interna a qual chamamos de núcleo. Sabendo disso, podemos começar a pensar como é possível através dos sensores, estudados na oficina anterior, realizar um mapeamento de todos os adesivos do cubo.

Exercício

Se temos um sensor que lê uma cor, qual é a solução de engenharia mais inteligente e barata para escanear todo o cubo mágico?

2. Sensoriamento

Após descobrir qual será a disposição dos sensores, inicia-se para o processo de sensoriamento. Esse processo irá funcionar como se fosse o olho do robô e irá nos gerar uma matriz

das cores de cada um dos adesivos e suas respectivas posições. Primeiramente, é definida uma letra para representar cada uma das cores lidas pelo sensor conforme mostra a Figura 2. E uma ordem de leitura que será realizada pelos nossos sensores, como é visto na Figura 3



Figura 2: Letras associadas a cada cor

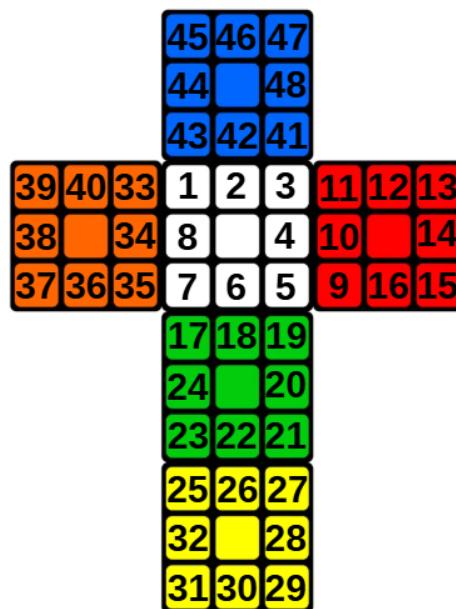


Figura 3: Ordem do sensoriamento

O sensoriamento realizado seguindo a ordem da figura acima, irá gerar 6 matrizes, uma para cada face, com 8 letras representando as cores contidas em cada um dos adesivos contidos naquela face.

EX: Na camada de cima (branca) teremos a matriz (*string*) [B,O,R,W,W,O,Y,G]

Note: Uma boa estratégia para verificar se a coleta das cores ocorreu corretamente, é verificar se cada uma das cores apareceu 8 vezes.

Exercício

Embaralhe o cubo aleatoriamente. Utilizando a notação de cores (W, R, G, Y, O, B) e o protocolo de ordem que definimos, escreva a *string* completa que representa o estado atual de todas as 48 peças .

3. Comunicação Serial

Tendo em mãos a *string* completa do estado do cubo, o arduino precisa então enviar a leitura obtida para o computador. Como é possível enviar apenas valores lógicos de 0 (baixo) ou 1 (alto), temos que encontrar um jeito de codificar nossa *string* em uma sequência enviável. Para isso, temos o padrão ASCII. O padrão relaciona um carácter a um número. Assim, quando o computador lê o número, ele faz a conversão para o respectivo carácter. Por exemplo, temos para a letra W o valor 87. Convertendo 87 para binário, temos 0101 0111. Que é uma sequência enviável e pode ser transferida do arduino para o computador. A figura 4 mostra a codificação do ASCII e a figura 5 mostra as sequências relevantes para a codificação do cubo.

ASCII/UTF-8 Comunicação por Bytes

ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter
32	space	48	0	64	@	80	P	96	~	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	\$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

Figura 4: Tabela de representação ASCII

W	→	0101 0111
R	→	0101 0010
G	→	0100 0111
Y	→	0101 1001
O	→	0100 1111
B	→	0100 0010

Figura 5: Letras traduzidas seguindo a tabela ASCII

Mas agora temos outro problema, como enviar a sequência? Uma maneira de realizar isso, é a chamada comunicação serial. Como o nome diz, ela envolve enviar serialmente, ou seja, um por um, uma sequência de bits para o computador. Cada bit é recebido individualmente, salvo, e então próximo é recebido. Isso pode ser observado com mais clareza na Figura 6. Nela temos que a sequência 1100110 é enviada bit por bit a cada subida do sinal de *clock*.

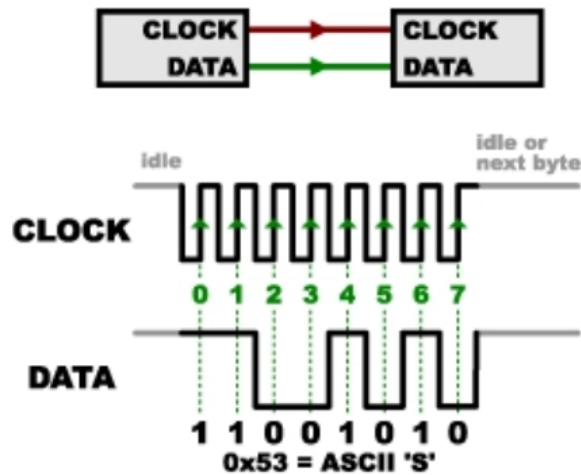


Figura 6: Comunicação serial

Exercício

Converte o início da *string* de 48 caracteres que representa o estado do cubo para a sua representação binária (em bytes), conforme o protocolo de codificação serial (ASCII/UTF-8). Mostre a sequência de 0s e 1s que o Python lerá.

4. Resumo da Etapa

A etapa de leitura pode ser bem resumida através da parte do arduino da Figura 7. Nela podemos ver a sequência de processos realizados para coletarmos toda a informação necessária para enfim adentrar na parte da solução do cubo mágico, que será proposta na linguagem Python.

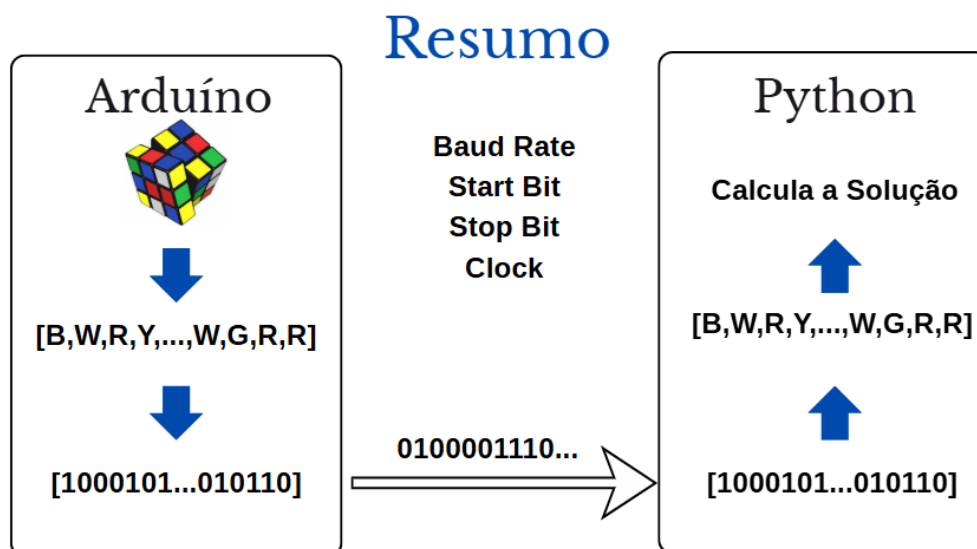


Figura 7: Resumo do processo de obtenção das cores e transmissão dos dados para manipulação em Python

Etapa 2: Métodos de Solução do Cubo Mágico

O cubo mágico (ou cubo de Rubik) foi inventado pelo professor de arquitetura húngaro Ernő Rubik. Ele o construiu para utilizá-lo como uma ferramenta interessante e diferente de ensino para auxiliar seus alunos a compreenderem relações espaciais e movimentos tridimensionais. A seguir iremos dissertar um pouco sobre alguns dos diversos métodos existentes para se resolver o rubik's cube.

1. Método de Camadas

No primeiro contato com a resolução do cubo mágico, é comum optar pelo método de camadas devido à sua simplicidade e abordagem intuitiva. Na Figura 8, estão ilustrados os principais passos dessa técnica. A estratégia consiste em resolver o cubo camada por camada, iniciando pela inferior e avançando progressivamente até a superior.

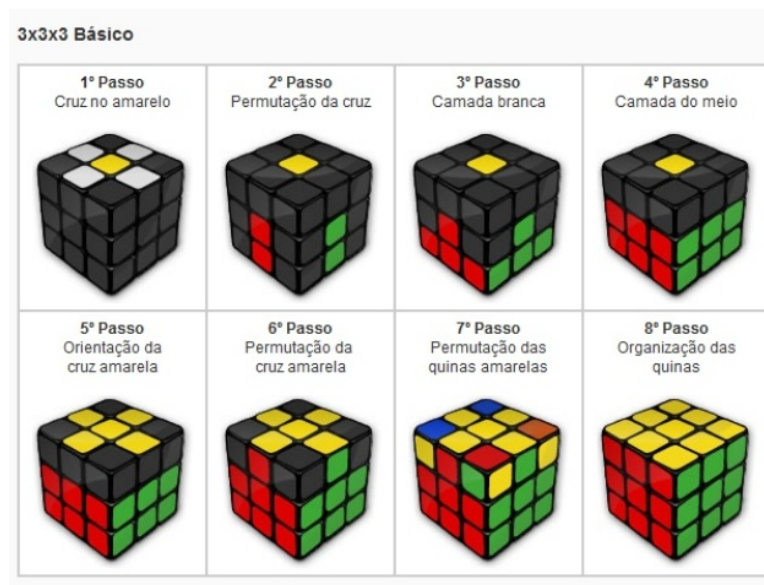


Figura 8: Método de camadas

2. Método Kociemba

O método de Kociemba, associado ao conceito do “Número de Deus”, é altamente otimizado para minimizar a quantidade de movimentos necessários para resolver o cubo mágico. Trata-se do método mais complexo abordado neste roteiro, e por isso não será explorado em profundidade.

Seu algoritmo inicia com uma série de movimentos que têm como objetivo transformar o estado inicial do cubo em uma configuração intermediária simplificada. A partir dessa configuração, a resolução é concluída utilizando um conjunto restrito de movimentos.

Esse método é capaz de gerar soluções com, no máximo, 20 movimentos, valor que representa o Número de Deus. Apesar de oferecer extrema eficiência na resolução, o algoritmo de Kociemba é tecnicamente sofisticado e de difícil compreensão, o que torna sua implementação e estudo mais desafiadores.

3. Método Vendado M2/OP

Este é o método usado para resolver “vendado”, e é o método que vamos usar no robô. A grande vantagem é que não precisamos atualizar o estado do cubo durante a solução: lê-se o cubo uma vez e pronto.

O nome vem das duas partes: M2 resolve os meios, e OP resolve as quinas. A ideia geral é simples: definimos um *buffer* e um *target*. Fazemos um *setup* para levar a peça alvo até o *target*, aplicamos o algoritmo (M2 ou OP), e desfazemos o *setup*. Repetindo isso, vamos colocando cada peça no seu lugar.

Essa lógica de troca de peças será melhor explicado em comutadores.

4. Notação oficial dos Cubos Mágicos

Para facilitar a comunicação e compreensão dos cubistas, foi desenvolvida uma notação muito intuitiva dos movimentos possíveis em um cubo mágico. Vamos listar um a um os movimentos possíveis:

F (frente), R (direita), L (esquerda), B (trás), U (cima), D (baixo) e M (meio vertical).

Sem apóstrofo é horário. Com apóstrofo (ex: R') é anti-horário. Com “2” (ex: U2) é o mesmo movimento duas vezes. A Figura 9 ilustra esses movimentos.

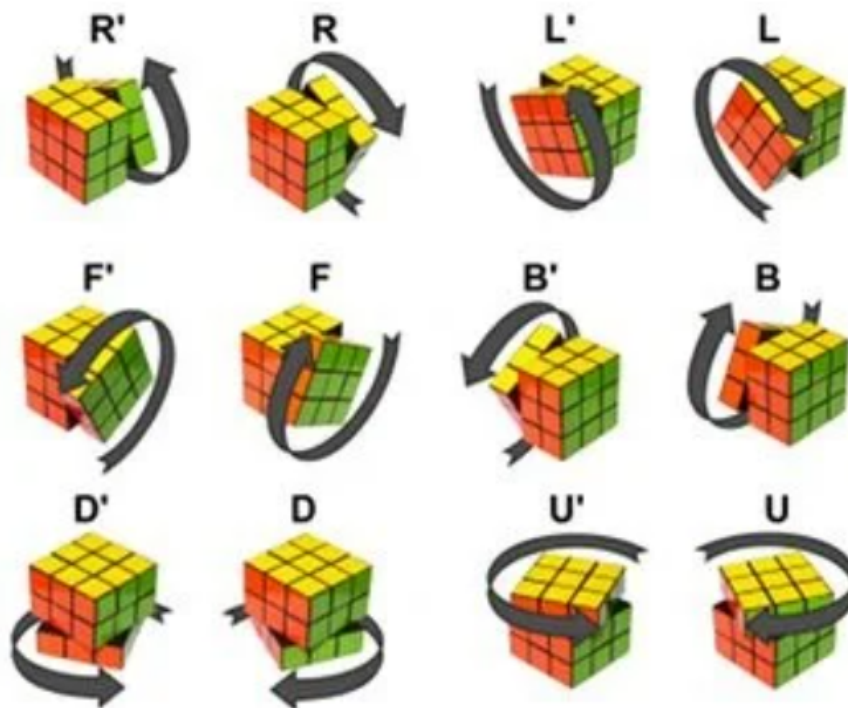


Figura 9: Possíveis movimentos no cubo

Exercício

Com o cubo mágico resolvido tente executar a seguinte sequência de movimentos: R U R' F' R U R' U' R' F R2 U' R' U'

5. Lógica de comutadores

Comutadores são a base do método vendado. Eles permitem trocar peças específicas sem bagunçar o resto do cubo. No M2/OP usamos semi-comutadores o tempo todo.

Matematicamente, um comutador é escrito como $[A B A' B']$, onde A e B são sequências de movimentos, e A' e B' são seus inversos.

Exemplo:

$$\begin{aligned}A &= U' \\B &= R'D'R \\A' &= U \\B' &= R'DR\end{aligned}$$

Logo, o comutador completo é: $[U' R' D' R U R' D R]$.

Exercício

Embaralhe o cubo com a seguinte sequência:

(F2 D' L2 D' L2 D2 F2 U' R2 U2 L' U R2 U' L U')

Tente achar um comutador que resolva esse caso. **Dica:** Use o exemplo anterior.

Resolvemos o cubo com uma sequência de comutadores, como pode ser visto na Figura 10.

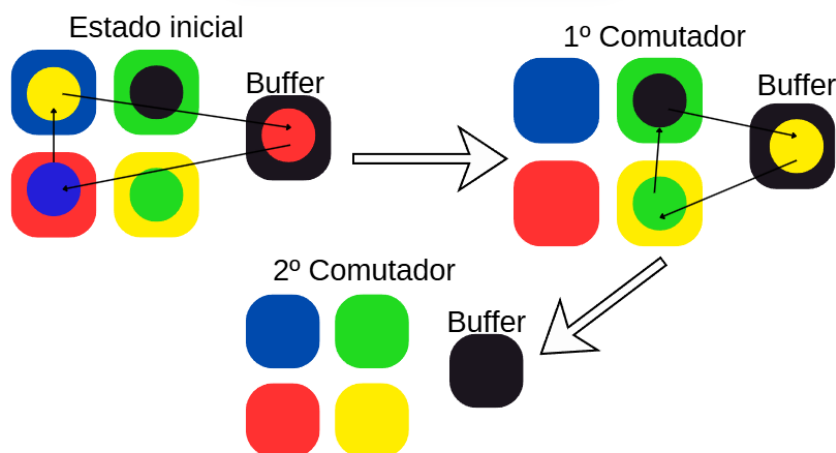


Figura 10: Lógica de comutadores

6. História

Com toda essa informação sobre o cubo, podemos começar a trabalhar com a história para resolver o cubo. A história, no método vendado, é nada mais nada menos do que uma sequência de letras que nos permite formar palavras e gerar uma história para facilitar uma memorização da sequência de movimentos que deverá ser seguida. Para isso mapeamos todos os meios e quinas do cubo com diferentes letras, para indicar qual peça deverá ser comutada na sequência. Na Figura 11 temos uma organização padrão entre os cubistas e um exemplo.

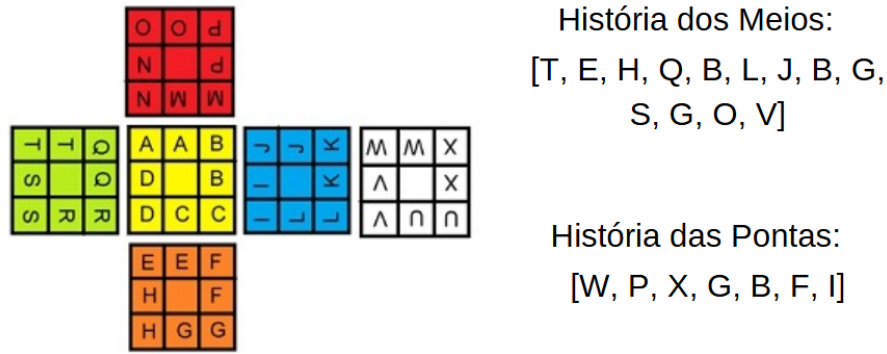


Figura 11: Mapeamento do cubo em letras

Utilizaremos a história em nosso algoritmo desenvolvido em Python para guiar a resolução do cubo nele, seguindo exatamente a mesma lógica seguida pelos cubistas do método vendado. Dessa forma, o código irá ir comutando sequencialmente as peças até chegar em uma sequência de movimentos que irá resolver o cubo.

Etapa 3: Comunicação com os motores

Após a obtenção dos movimentos necessários para solução do cubo, temos a missão de transferir a mensagem para os motores. Essa etapa funcionará como um cérebro se comunicando com os braços de uma pessoa para realizar uma ação.

Assim como na comunicação dos sensores, iremos utilizar a tabela ASCII vista na 4. Esse processo final pode ser identificado na Figura 12.

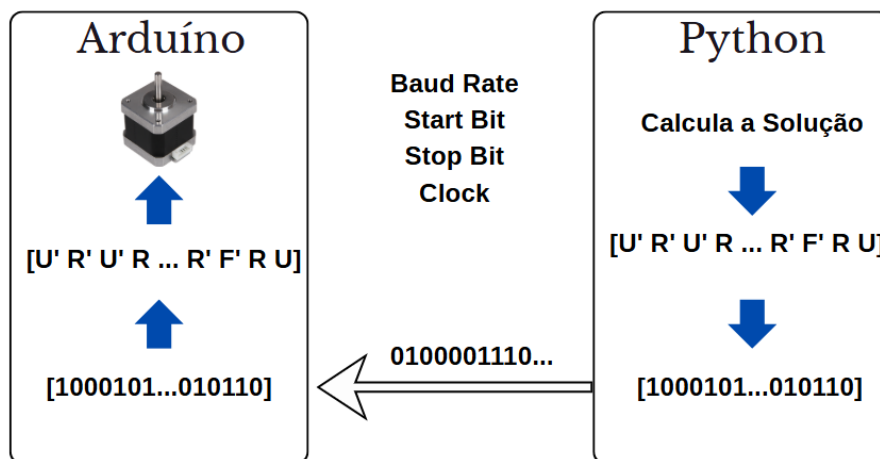


Figura 12: Transmissão da solução em Python para os motores